

COM3110/4115:

Text Processing

*Text Compression: Arithmetic coding and Further
Topics in Symbolwise Methods*

Rob Gaizauskas

Department of Computer Science
University of Sheffield

- Introduction to Text Compression
 - ◇ Lossless vs lossy compression
 - ◇ Dictionary vs Symbolwise methods
 - ◇ Static vs Semi-static vs Adaptive methods
- Symbolwise methods
 - ◇ Introduction to Symbolwise methods
 - ◇ Coding
 - Huffman Coding
 - Arithmetic Coding
 - ◇ Further topics in Symbolwise methods
- Dictionary Methods
 - ◇ LZ77 and gzip
- Further topics:
 - ◇ Synchronisation
 - ◇ Performance Issues

- Introduction to Text Compression
 - ◇ Lossless vs lossy compression
 - ◇ Dictionary vs Symbolwise methods
 - ◇ Static vs Semi-static vs Adaptive methods
- Symbolwise methods
 - ◇ Introduction to Symbolwise methods
 - ◇ Coding
 - Huffman Coding
 - Arithmetic Coding
 - ◇ Further topics in Symbolwise methods
- Dictionary Methods
 - ◇ LZ77 and gzip
- Further topics:
 - ◇ Synchronisation
 - ◇ Performance Issues



This lecture

Arithmetic Coding

- Arithmetic coding allows excellent compression
 - ◇ can code arbitrarily close to the entropy
 - ◇ hence is *optimal* terms of compression
- Wins over Huffman coding if distribution is *very skewed*
 - e.g. for two letter alphabet $\{a, b\}$ where $Pr[a] = .99$ and $Pr[b] = 0.01$
 - a can be coded in $-\log_2 Pr[s] = 0.015$ bits
 - Huffman coding, however, requires at least one-bit/symbol
 - same not true for arithmetic coding
- Arithmetic coding suitable for sophisticated adaptive models
- Disadvantages:
 - ◇ slower than Huffman coding
 - ◇ not easy to start decoding in middle of compressed stream
 - ◇ hence less suitable for full-text retrieval
(where random access to compressed text may be needed)
- *Thus*: Huffman most useful for text; arithmetic coding for images

Arithmetic Coding (ctd)

- Output of arithmetic coder is a stream of bits, but think of it as a fractional binary number between 0 and 1:

e.g. 0.1011001



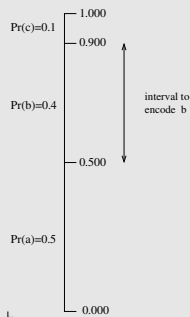
$\frac{1}{2} + \frac{1}{8} + \frac{1}{16} + \frac{1}{128} = \sim 0.695$

$\frac{1}{2} \quad \frac{1}{4} \quad \frac{1}{8} \quad \frac{1}{16} \quad \frac{1}{32} \quad \frac{1}{64} \quad \frac{1}{128}$

- ◇ can drop the "0." prefix, as same on all outputs
- Suppose we have a ternary alphabet $\{a, b, c\}$, and want to compress text *bab*
 - ◇ assume (static) model: $\Pr(a)=0.5 \quad \Pr(b)=0.4 \quad \Pr(c)=0.1$
- Arithmetic coder stores two numbers – *high* and *low* – representing a subinterval of $[0, 1]$ used to code next symbol
 - ◇ initially *high* = 1 and *low* = 0

Arithmetic Coding (ctd)

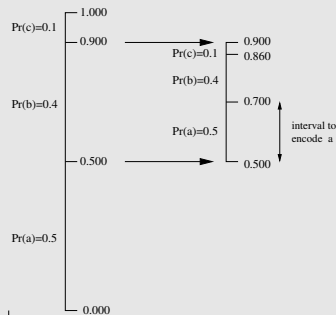
- Range between *high* and *low* sub-divided according to probability distribution of model: sub-intervals allocated for coding each symbol



- The **coding step** involves resetting the *high/low* values to **narrow** the recorded interval
 - ◇ here, to code *b*, set *high* = 0.9 and *low* = 0.5

Arithmetic Coding (ctd)

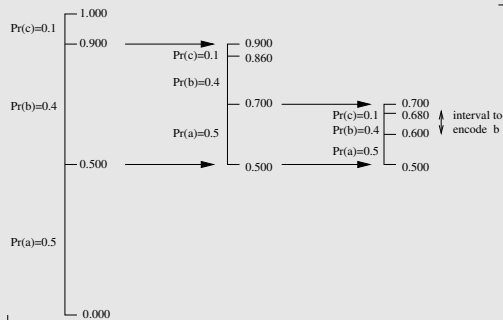
- Coding continues by sub-dividing new current interval (between *high/low*) and narrowing to sub-interval for next coded symbol



◇ here, set *high* = 0.7 and *low* = 0.5, to code next symbol *a*

Arithmetic Coding (ctd)

- Repeat process to code third symbol *b*:



- ◇ set *high* = 0.68 and *low* = 0.6, to code final symbol *b*
- ◇ final interval represents **full** input *bab*

Arithmetic Coding (ctd)

- Encoder has processed full input *bab*
 - ◊ coding process has narrowed interval so: $high = 0.68$, $low = 0.6$
- Encoder now transmits this content by outputting *any number* in the range between *high* and *low*
 - ◊ choose number with *shortest code* (of course!)
 - e.g. choose 101 (0.625), not 10101 (~ 0.656), though both in range
- Transmitted code, plus model, is sufficient for decoding
 - ◊ decoder simulates steps in encoding process
 - ◊ and as it does, recovers encoded content
- The smaller the final interval is, the more bits that will be needed to specify a number that falls within it
 - e.g. for our example, low probability input *ccc* has final interval width 0.001, and requires more bits to transmit (c.f. input *aaa* with interval width 0.125)

Arithmetic Coding (ctd)

- Intuitively, method works because high probability events narrow the interval much less than low probability events do
- The number of bits required is proportional to the negative logarithm of the size of the interval
 - ◇ the interval size is the product of the probabilities of the coded symbols
 - ◇ the logarithm of this quantity is the sum of the logarithms of the individual probabilities
 - ◇ symbol s of probability $P[s]$ contributes $-\log_2 P[s]$ bits to output
 - this is equivalent to the symbol's information content
- Hence method is near-optimal
 - ◇ code size identical to the theoretical bound given by the entropy
 - ◇ high probability symbols can be coded in a fraction of a bit

Arithmetic Coding (ctd)

- Method is limited in practice by:
 - ◇ need to transmit, eventually, a whole number of bits/bytes
 - ◇ limited precision arithmetic
- As described, method produces no output until encoding complete
i.e. until all input processed
- In practice, possible to output bits *during* coding
 - ◇ avoids having to work with higher and higher precision numbers
 - ◇ key observation: when range is *small*, *high/low* have *common prefix*
e.g. if range is *high* = 0.6667, *low* = 0.6334 might:
 - output bits for prefix (here 6)
 - reset range as *high* = 0.667, *low* = 0.334
 - ◇ allows output to be generated *incrementally*

Arithmetic Coding (ctd)

- Arithmetic coding more commonly used with *adaptive modelling*
 - ◊ probabilities used based on counts observed in text
- Consider earlier example with alphabet $\{a, b, c\}$, coding input *bab*:
 - ◊ initialise counts to 1 (avoid zero-frequency problem)
 - ◊ initial probabilities then: $Pr(a) = \frac{1}{3}$ $Pr(b) = \frac{1}{3}$ $Pr(c) = \frac{1}{3}$
 - ◊ this model used to code first character *b* — then counts updated
 - ◊ updated model then: $Pr(a) = \frac{1}{4}$ $Pr(b) = \frac{2}{4}$ $Pr(c) = \frac{1}{4}$
 - ◊ after 2nd char *a* coded, model: $Pr(a) = \frac{2}{5}$ $Pr(b) = \frac{2}{5}$ $Pr(c) = \frac{1}{5}$
 - ◊ after 3rd char *b* coded, model: $Pr(a) = \frac{2}{6}$ $Pr(b) = \frac{3}{6}$ $Pr(c) = \frac{1}{6}$
 - ◊ and so on ...
 - ◊ see Witten *et al.* for a worked example
- As before, given output, decoder can simulate the encoding process, including all the counting/model updates

- Introduction to Text Compression
 - ◇ Lossless vs lossy compression
 - ◇ Dictionary vs Symbolwise methods
 - ◇ Static vs Semi-static vs Adaptive methods
- Symbolwise methods
 - ◇ Introduction to Symbolwise methods
 - ◇ Coding
 - Huffman Coding
 - Arithmetic Coding
 - ◇ Further topics in Symbolwise methods
- Dictionary Methods
 - ◇ LZ77 and gzip
- Further topics:
 - ◇ Synchronisation
 - ◇ Performance Issues



This lecture

Symbolwise Models: Non-zero order models

- As we have seen, **symbolwise methods** work by estimating the probabilities of symbols (characters/words) and coding one symbol at a time using shorter codewords for the more likely symbols
- So far, only *specific symbolwise model* considered (in examples, etc) has been **zero order character models**
 - i.e. character models that do not consider left context
 - ◇ not a serious contender for effective text compression
- PPM is a character-based symbolwise model that uses left context
 - ◇ PPM = *prediction by partial matching*
 - ◇ determines probabilities in a given (finite) context
 - ◇ **but** length of context may vary depending on contexts previously seen
 - ◇ obtains arguably the best compression performance

Symbolwise Models: Word-based Models

- So far have largely assumed “symbols” are characters
- In word-based models, symbols are “words” (alphanumeric strings) and “non-words” (white-space and punctuation)
- Documents are assumed to consist of strictly alternating words and non-words and (typically)
 - ◊ one zero-order model is built for words
 - ◊ another zero-order model is built for nonwords
- If an adaptive model is used, a mechanism is needed for previously unseen words/nonwords
 - e.g. send an escape symbol then spell out the word character by character, using a zero-order model of characters

Symbolwise Models: Word-based Models (ctd)

- “Parsing” text into words-nonwords raises a number of issues
 - ◇ how is punctuation that is part of a word (hyphens, apostrophes) to be handled?
 - ◇ should every numeric string be a separate word?
 - can lead to huge numbers of words that occur once only
 - ◇ what about ideographic languages?
 - segmenting text into words much more challenging for these
- Word-based models can yield large numbers of symbols
 - ◇ efficient data structure for model important
 - canonical Huffman code good for static/semi-static versions
- Can achieve compression performance close to PPM
 - ◇ with (semi-) static models supports random access

- R. Baeza-Yates and B. Ribeiro-Neto (2011). Modern Information Retrieval: The Concepts and Technology Behind Search (ACM Press Books), 2nd edition. Chapter 6.8.
- I. H. Witten, A. Moffat, T. C. Bell. Managing Gigabytes: Compressing and Indexing Documents and Images, 2nd ed. Morgan Kaufmann. 1999.
- Nam Phamdo. Theory of Data Compression.
www.data-compression.com/theory.html